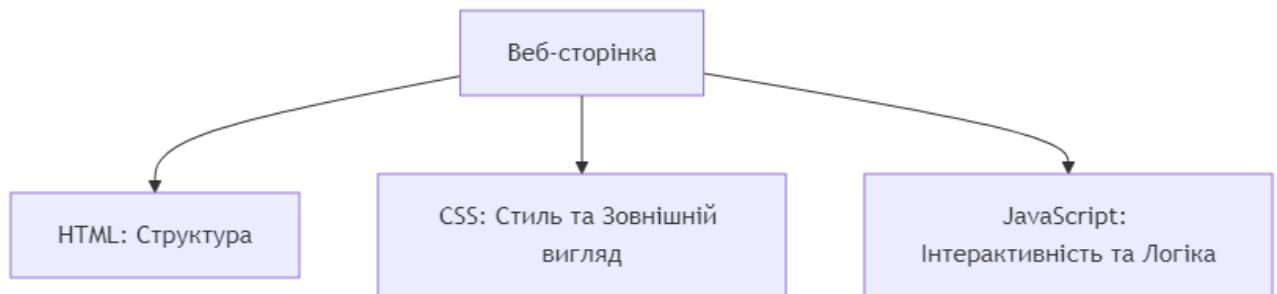


Основи мови JavaScript та маніпуляція DOM

1. Вступ до JavaScript та його роль у сучасній веб-розробці

JavaScript — це високорівнева, інтерпретована (або JIT-компільована), багатопарадигмова мова програмування з динамічною типізацією. JavaScript є ключовим елементом сучасної веб-екосистеми, забезпечуючи інтерактивність та динамічну поведінку веб-сторінок.



Історія створення та стандартизація (ECMAScript)

Мова JavaScript була створена Бренданом Ейхом (Brendan Eich) у 1995 році за 10 днів під час його роботи в компанії Netscape Communications. Початкова назва мови — **Mocha**, пізніше її змінили на **LiveScript**, і, нарешті, з маркетингових міркувань (через зростаючу популярність Java) — на **JavaScript**.

Для запобігання фрагментації мови та забезпечення сумісності між різними браузерами було прийнято рішення стандартизувати її. Міжнародна організація **Ecma International** випустила специфікацію **ECMAScript (ECMA-262)**.

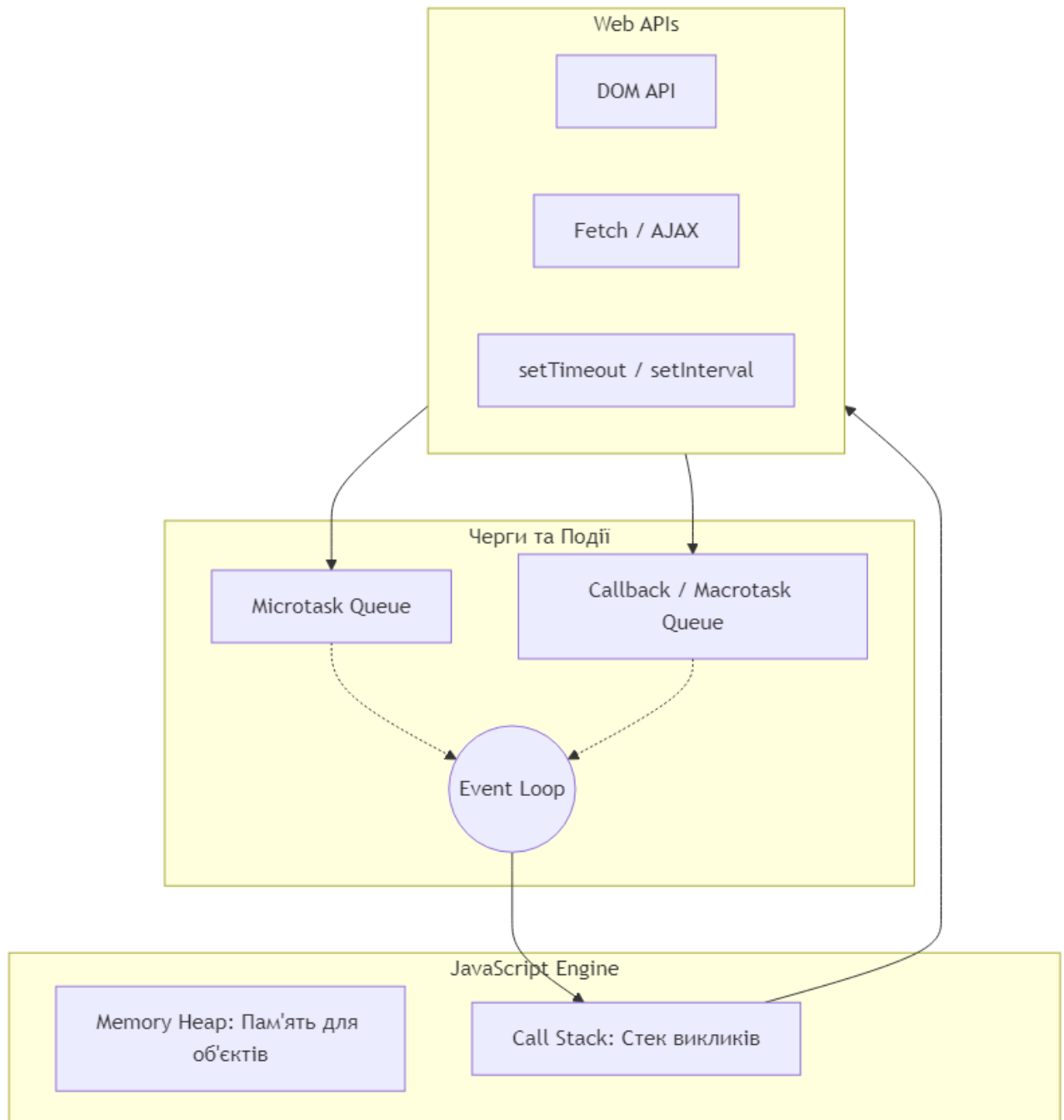
Ключові етапи розвитку ECMAScript:

- **ES3 (1999):** Додано регулярні вирази, обробку винятків (`try...catch`).
- **ES5 (2009):** Додано строгого режиму ("`use strict`"), JSON, методи масивів (`map`, `filter`, `reduce`).
- **ES6 / ECMAScript 2015:** Переломний реліз. Додано `let/const`, стрілкові функції, класи, проміси, модулі, шаблони рядків.
- **Щорічні оновлення (ES2016+):** Випуск оновлень щороку (`async/await`, `optional chaining`, `nullish coalescing` тощо).

2. Середовище виконання JavaScript та Event Loop

JavaScript працює всередині середовища виконання (Runtime Environment), найвідомішими з яких є браузери (Google Chrome, Mozilla Firefox, Safari) та серверне середовище **Node.js**.

Компоненти середовища виконання браузера



Call Stack (Стек викликів): Однопоточкова структура даних, яка відстежує поточне місце виконання коду. Реалізує принцип LIFO (Last In, First Out).

- **Memory Heap (Купа пам'яті):** Неструктурована область пам'яті, де виділяється місце під об'єкти, масиви та функції.
- **Web APIs:** Надаються браузером (не є частиною ECMAScript). Включають роботу з DOM, `fetch`, таймери (`setTimeout`).
- **Event Loop (Цикл подій):** Механізм, який безперервно перевіряє Стек викликів та Черги задач. Якщо Стек порожній, Event Loop бере першу задачу з Черги та відправляє її в Стек.

3. Синтаксис мови JavaScript: Змінні, Типи даних та Оператори

Оголошення змінних: `var`, `let`, `const`

До виходу стандарту ES6 основним способом оголошення змінних було ключеве слово `var`. Сучасний JavaScript використовує `let` та `const`.

```
// Старий підхід (var)
var title = "JavaScript";

// Сучасний підхід (let та const)
let count = 10;
count = 11; // Дозволено зміну значення

const PI = 3.14159;
// PI = 3.14; // Uncaught TypeError: Assignment to constant variable.
```

Порівняльна таблиця типів оголошень:

Характеристика	var	let	const
Область видимості (Scope)	Функціональна	Блочна {}	Блочна {}
Підняття (Hoisting)	Так (ініціалізується як undefined)	Так (перебуває у TDZ)	Так (перебуває у TDZ)
Переоголошення	Дозволено	Заборонено	Заборонено
Мутабельність	Дозволено зміну	Дозволено зміну	Заборонено перезапис

Що таке Hoisting (Підняття)?

Hoisting (підняття) — це внутрішній механізм у JavaScript, за якого оголошення змінних, функцій або класів "переміщуються" (піднімаються) у початок їхньої області видимості (скрипту або функції) під час фази компіляції перед безпосереднім виконанням коду.

Фактично код фізично не змінює свій порядок — це інтерпретатор JavaScript сканує код перед виконанням, виділяє пам'ять під оголошення та запам'ятовує їх.

Як Hoisting працює для різних конструкцій:

1. Оголошення змінних через `var`

Змінні, оголошені через `var`, піднімаються у початок своєї області видимості та ініціалізуються значенням `undefined`.

```
console.log(myVar); // undefined (помилки немає, але значення ще немає)
var myVar = 10;
console.log(myVar); // 10
```

Як це насправді бачить JavaScript:

```
var myVar;           // Оголошення піднято й ініціалізовано значенням undefined
console.log(myVar); // undefined
myVar = 10;          // Присвоєння значення залишається на своєму місці
console.log(myVar); // 10
```

2. Оголошення змінних через `let` та `const`

Змінні, оголошені за допомогою `let` та `const`, також піднімаються, але не ініціалізуються.

Проміжок часу між початком області видимості та точкою оголошення змінної називається **Тимчасовою мертвою зоною (Temporal Dead Zone, TDZ)**. Якщо звернутися до такої змінної до рядка її оголошення, ви викличете помилку `ReferenceError`.

```
console.log(myLet); // Uncaught ReferenceError: Cannot access 'myLet' before
initialization
let myLet = 20;
```

3. Оголошення функцій (Function Declaration)

Оголошення функцій через `Function Declaration` піднімаються **повністю** — піднімається як назва функції, так і її тіло (реалізація). Тому функцію можна викликати у коді вище того місця, де вона написана.

```
sayHello(); // Працює! "Привіт!"

function sayHello() {
  console.log("Привіт!");
}
```

4. Функціональні вирази (Function Expression) та Стрілкові функції

Якщо функція оголошується через змінну (`Function Expression` або `Arrow Function`), то до неї застосовуються правила підняття тієї змінної, у яку вона записана.

```
// З var:
sayHi(); // TypeError: sayHi is not a function (бо значення змінної sayHi ще
undefined)

var sayHi = function() {
  console.log("Hi!");
};

// З const / let:
sayBye(); // ReferenceError: Cannot access 'sayBye' before initialization

const sayBye = () => {
  console.log("Bye!");
};
```

Мутабельність (Mutable / Mutability) у програмуванні та JavaScript зокрема — це властивість об'єкта або структури даних **змінювати свій внутрішній стан або вміст після створення**, без створення нового об'єкта в пам'яті.

Протилежним поняттям є **Імутабельність (Immutable / Immutability)** — незмінність даних після їх створення.

Мутабельні та Імутабельні типи в JavaScript

В екосистемі JavaScript типи даних чітко розділені за цією ознакою:

- **Імутабельні (незмінні)** — усі примітивні типи даних: `Number`, `String`, `Boolean`, `Null`, `Undefined`, `Symbol`, `BigInt`.
- **Мутабельні (змінні)** — складні типи даних (об'єкти): `Object`, `Array`, `Function`, `Map`, `Set`, `Date` тощо.

Приклад 1: Імутабельність примітивів (Рядки)

Коли ви застосовуєте методи до рядка, початковий рядок не змінюється — метод повертає **новий** рядок.

```
let str = "hello";
str.toUpperCase(); // повертає "HELLO", але сам str не змінився!

console.log(str); // "hello" (початкове значення збереглося)

// Щоб зберегти результат, потрібно перезаписати змінну:
str = str.toUpperCase(); // створюється новий рядок в пам'яті
```

Приклад 2: Мутабельність складних типів (Масиви та Об'єкти)

При зміні властивості об'єкта або елемента масиву змінюється сам об'єкт у пам'яті за тим самим посиланням.

```
// Мутабельність об'єкта
const user = { name: "Олена", age: 25 };
user.age = 26; // Початковий об'єкт змінено!
console.log(user); // { name: "Олена", age: 26 }

// Мутабельність масиву
const numbers = [1, 2, 3];
numbers.push(4); // Метод push мутує (змінює) початковий масив
console.log(numbers); // [1, 2, 3, 4]
```

Зв'язок з ключевим словом `const`

- **`const` забороняє перезапис посилання (змінної):**

```
const obj = { title: "JS" };
obj = { title: "React" }; // TypeError: Assignment to constant variable.
```

- **Але `const` НЕ робить сам об'єкт імутабельним:**

```
const obj = { title: "JS" };
obj.title = "React"; // Дозволено! Вміст об'єкта змінено.
console.log(obj.title); // "React"
```

Важливий нюанс: Передача за посиланням

Оскільки мутабельні об'єкти передаються за посиланням, зміна об'єкта через одну змінну вплине на всі інші змінні, які посилаються на цей самий об'єкт:

```
const person1 = { name: "Іван" };
const person2 = person1; // person2 посилається на той самий об'єкт

person2.name = "Петро"; // Мутуємо об'єкт через person2

console.log(person1.name); // "Петро" (змінилося і в person1!)
```

Як зробити об'єкт імутабельним у JavaScript?

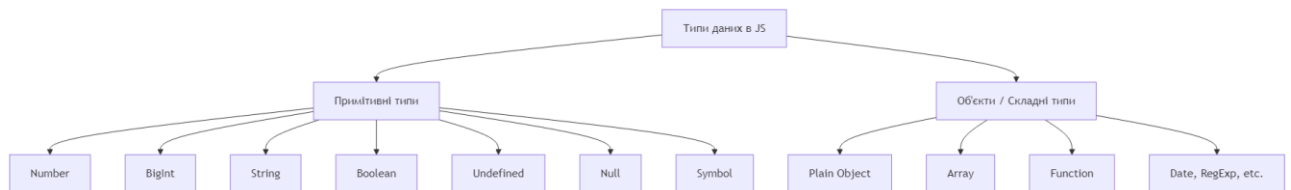
За потреби заморозити об'єкт від подальших змін використовують метод `Object.freeze()`:

```
const config = Object.freeze({
  apiEndpoint: "https://api.example.com",
  port: 8080
});
```

```
config.port = 3000; // Помилка у суворому режимі ('use strict') або просто ігнорується
console.log(config.port); // 8080 (значення не змінилося)
```

Типи даних в JavaScript

JavaScript є мовою з динамічною типізацією. Типи даних діляться на дві категорії: **Примітиви** та **Об'єкти (Складні типи)**.



1. Примітивні типи (передаються за значенням):

- **Number:** Числа із плаваючою крапкою подвійної точності (IEEE 754).
- **BigInt:** Для цілих чисел довільної величини (10n).
- **String:** Текстові дані.
- **Boolean:** Логічний тип (true / false).
- **Undefined:** Змінна оголошена, але значення не присвоєно.
- **Null:** Спеціальне значення, що вказує на відсутність значення.
- **Symbol:** Унікальний та незмінний ідентифікатор.

2. Складні типи (передаються за посиланням):

- **Object:** Структура для зберігання колекцій даних у вигляді "ключ-значення".

4. Фундаментальні концепції: Функції, Області видимості та Замикання

Типи оголошення функцій

Функції в JavaScript є об'єктами першого класу (First-class citizens) — їх можна присвоювати змінним, передавати як аргументи та повертати з інших функцій.

Function Declaration

```
function calculateArea(width, height) {
    return width * height;
}
// Підтримує hoisting (можна викликати до оголошення у коді)
```

Function Expression

```
const calculateArea = function(width, height) {
    return width * height;
};
```

Arrow Functions (Стрілкові функції, ES6)

Стрілкові функції (Arrow Functions) — це компактний синтаксис для оголошення функцій, який з'явився у стандарті ECMAScript 2015 (ES6).

Вони записуються за допомогою синтаксису `() => {}` (стрілки):

```
// Класична функція (Function Expression)
const multiply = function(a, b) {
  return a * b;
};
```

```
// Стрілкова функція (Arrow Function)
const multiply = (a, b) => a * b;
```

Основні особливості та синтаксис:

1. Скорочений синтаксис (Implicit Return):

- Якщо тіло функції складається лише з одного виразу, фігурні дужки `{ }` та слово `return` можна опустити — значення повернеться автоматично:

```
const square = x => x * x; // повертає результат x * x
```

- Якщо параметр лише один, круглі дужки навколо нього також можна опустити:

```
const double = n => n * 2;
```

2. Відсутність власного контексту `this`:

- Це головна відмінність: стрілкові функції **не мають власного `this`**.
- Вони успадковують `this` з навколишнього (лексичного) середовища, де вони були оголошені. Це дуже зручно при використанні в обробниках подій або межах методів об'єктів/класів.

3. Не мають об'єкта `arguments`:

- У середині стрілкової функції немає спеціального псевдомасиву `arguments`. Замість цього використовують Rest-параметри `(...args)`.

4. Не можуть бути конструкторами:

- Стрілкові функції не можна викликати з оператором `new`, і вони не мають властивості `prototype`.

Замикання (Closures)

Замикання — це комбінація функції та лексичного середовища (Lexical Environment), в якому ця функція була оголошена. Замикання дозволяє внутрішній функції отримувати доступ до змінних із зовнішньої області видимості навіть після того, як зовнішня функція завершила своє виконання.

```
function createCounter() {
  let count = 0; // Приватна змінна

  return {
    increment() {
      count++;
      return count;
    },
    decrement() {
      count--;
      return count;
    },
    getValue() {
```

```

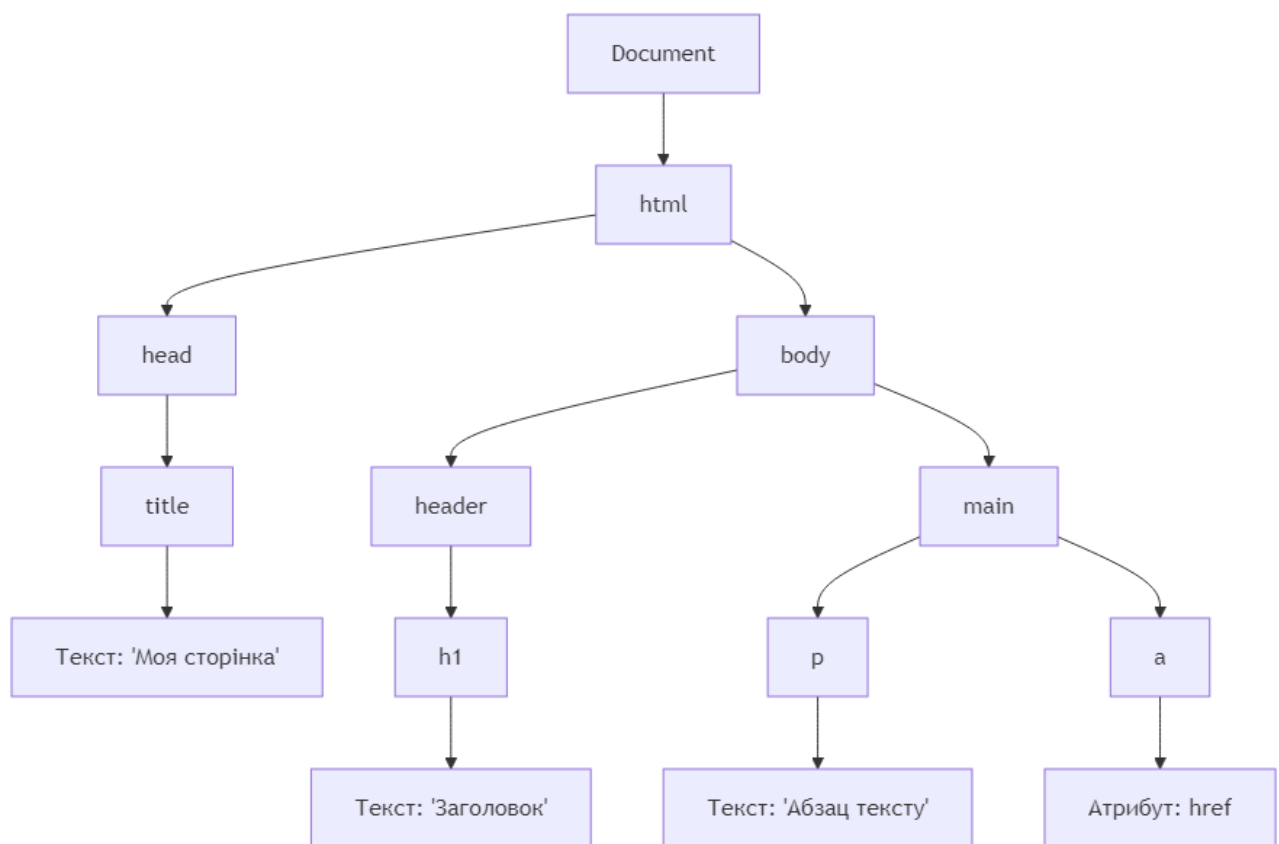
        return count;
    }
};
}

const counter = createCounter();
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.getValue());  // 2

```

5. Об'єктна модель документа (DOM): Структура та Дерево вузлів

DOM (Document Object Model) — це програмний інтерфейс (API) для HTML та XML документів. Він представляє структуру документа у вигляді древовидної моделі, де кожен елемент, атрибут та текст є окремим **вузлом (Node)**.



Типи вузлів DOM (Node Types)

Відповідно до специфікації **DOM (Document Object Model)**, кожен вузол у DOM-дереві має властивість `nodeType`, яка повертає числову константу (від 1 до 12).

Нижче наведено повний офіційний перелік усіх 12 типів вузлів DOM:

Повний перелік типів вузлів DOM (`nodeType`)

Числовий код (nodeType)	Строкова константа	Назва типу вузла	Опис та приклади
1	ELEMENT_NODE	Елемент	HTML або XML теги (наприклад, <div>, <p>, <a>).
2	ATTRIBUTE_NODE	Атрибут	Атрибути елементів (наприклад, class="card", id="main"). (У сучасному DOMAttr не вважається прямим дочірнім вузлом елемента)
3	TEXT_NODE	Текстовий вузол	Текст всередині елементів або між ними, включаючи пробіли та переноси рядків.
4	CDATA_SECTION_NODE	Секція CDATA	Блок тексту CDATA в XML-документах (наприклад, <![CDATA[. . .] >). Не використовується в HTML5.
5	ENTITY_REFERENCE_NODE	Посилання на сутність	Посилання на сутності в XML (застаріло в сучасних стандартах DOM).
6	ENTITY_NODE	Сутність	Оголошення сутності в XML/DTD (застаріло в сучасних стандартах DOM).
7	PROCESSING_INSTRUCTION_NODE	Інструкція обробки	Інструкції обробки в XML (наприклад, <?xml-stylesheet ...?>).

Числовий код (nodeType)	Строкова константа	Назва типу вузла	Опис та приклади
8	COMMENT_NODE	Коментар	Коментарі у вихідному коді (наприклад, <code><!-- Текст коментаря --></code>).
9	DOCUMENT_NODE	Документ	Корінь DOM-дерева (document).
10	DOCUMENT_TYPE_NODE	Тип документа	Оголошення <code><!DOCTYPE html></code> , яке визначає версію HTML/XML.
11	DOCUMENT_FRAGMENT_NODE	Фрагмент документа	Полегшений об'єкт <code>DocumentFragment</code> , який використовується як тимчасовий контейнер для маніпуляцій з групою вузлів.
12	NOTATION_NODE	Нотація	Оголошення нотацій у XML DTD (застаріло).

Приклад перевірки типу вузла в JavaScript:

Вузли можна перевіряти за допомогою числового значення або за допомогою відповідних констант об'єкта `Node`:

```
const element = document.querySelector('h1');

// Перевірка через числове значення:
if (element.nodeType === 1) {
  console.log('Це HTML-елемент');
}

// Перевірка через константу Node:
if (element.nodeType === Node.ELEMENT_NODE) {
  console.log('Це HTML-елемент');
}
```

6. Пошук та Навігація по DOM-дереву

Методи вибірки елементів (DOM Selection)

Сучасний JavaScript пропонує ефективні методи для пошуку елементів у DOM-дереві.

Старі класичні методи:

- `document.getElementById(id)` — повертає один елемент за його `id`.
- `document.getElementsByClassName(className)` — повертає `HTMLCollection`.
- `document.getElementsByTagName(tagName)` — повертає `HTMLCollection`.

Сучасні селектори (рекомендований підхід):

- `document.querySelector(selector)` — повертає перший елемент, що відповідає CSS-селектору.
- `document.querySelectorAll(selector)` — повертає статичний `NodeList` усіх елементів, що відповідають CSS-селектору.

```
// Пошук за id
const mainHeader = document.querySelector('#main-header');

// Пошук за класом
const activeButtons = document.querySelectorAll('.btn.active');

// Пошук складним CSS-селектором
const firstCardInput = document.querySelector('.card:first-child
input[type="text"]');
```

7. Модифікація DOM: Створення, Видалення та Редагування Елементів

Зміна вмісту та атрибутів

- `element.textContent` — безпечне зчитування та запис текстового вмісту (ігнорує HTML теги).
- `element.innerHTML` — зчитування та зміна HTML-структури всередині елемента. *(Увага: використання із неперевіреними даними створює ризик XSS-атак!).*

```
const title = document.querySelector('h1');
title.textContent = 'Новий заголовок'; // Безпечно

const container = document.querySelector('.container');
container.innerHTML = '<p>Динамічний абзац</p>'; // Інтерпретує як HTML
```

Створення та вставка елементів

Для запобігання зайвим повторним перемальовуванням (Reflow/Repaint) рекомендується використовувати фрагменти `DocumentFragment` при масовому додаванні елементів.

```
// Створення нового елемента
const newCard = document.createElement('div');
newCard.classList.add('card');

const cardTitle = document.createElement('h3');
cardTitle.textContent = 'Картка товару';
```

```
newCard.appendChild(cardTitle);

// Вставка у DOM
const grid = document.querySelector('.grid');
grid.appendChild(newCard); // Додає у кінець
```

Методи сучасної вставки:

- `node.append(...nodesOrStrings)` — додає елементи в кінець `node`.
- `node.prepend(...nodesOrStrings)` — додає елементи в початок `node`.
- `node.before(...nodesOrStrings)` — додає перед `node`.
- `node.after(...nodesOrStrings)` — додає після `node`.
- `node.remove()` — видаляє елемент з DOM.

8. Робота зі стилями та класами (CSSOM)

Управління CSS-класами (`classList`)

Об'єкт `classList` надає зручний інтерфейс для роботи з CSS-класами елемента:

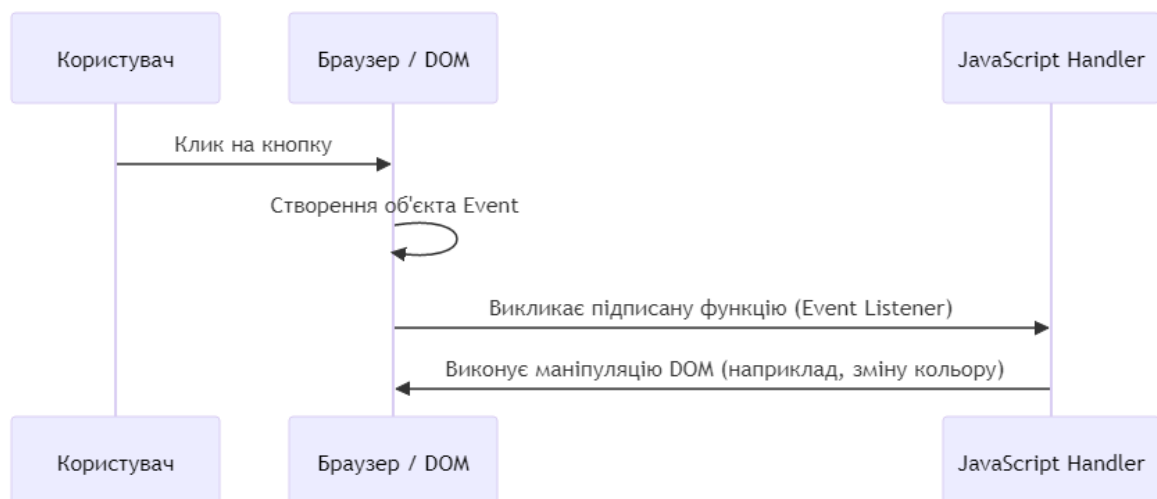
- `element.classList.add('class1', 'class2')` — додає класи.
- `element.classList.remove('class1')` — видаляє клас.
- `element.classList.toggle('active')` — перемикає клас (додає, якщо відсутній, і навпаки).
- `element.classList.contains('active')` — повертає `true`, якщо клас присутній.

Пряма маніпуляція `inline`-стилями

```
const box = document.querySelector('.box');
box.style.backgroundColor = '#f44336';
box.style.marginTop = '20px';
```

9. Події та Подієве програмування (Event Driven Architecture)

Події в браузері — це сигнали про те, що щось відбулося (клик миші, завантаження сторінки, натискання клавіші).



Реєстрація обробників подій

Основний та рекомендований метод додавання слухача подій — `addEventListener`.

```
const btn = document.querySelector('#submit-btn');

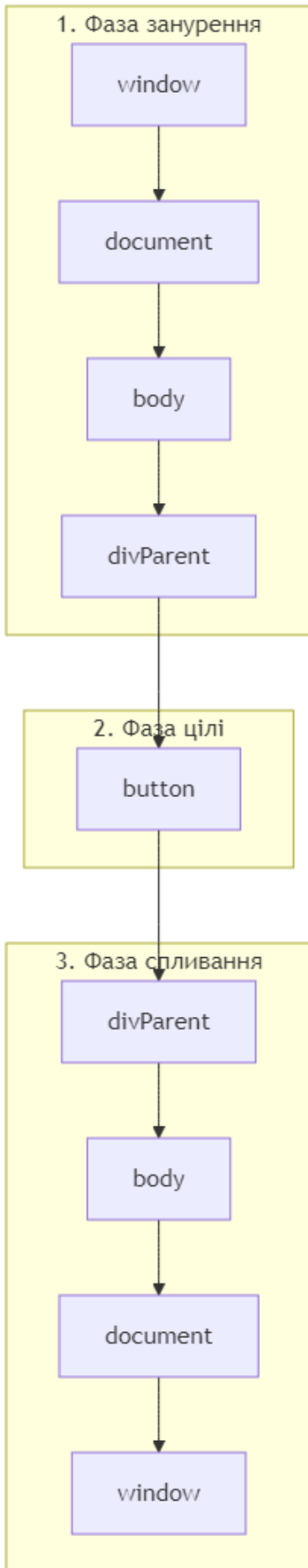
function handleClick(event) {
    console.log('Подія виникла на елементі:', event.target);
    console.log('Тип події:', event.type);
}

// Додавання слухача
btn.addEventListener('click', handleClick);

// Видалення слухача (вимагає посилання на той самий функціональний об'єкт)
btn.removeEventListener('click', handleClick);
```

10. Життєвий цикл події: Занурення, Ціль та Спливання (Event Bubbling)

Події в DOM проходять через три послідовні фази при настанні на будь-якому елементі.



1. **Фаза занурення (Capturing Phase):** Подія опускається від `window` вниз по DOM-дереву до цільового елемента.

2. **Фаза цілі (Target Phase):** Подія досягає цільового елемента (`event.target`).
3. **Фаза спливання (Bubbling Phase):** Подія піднімається вгору від цільового елемента назад до `window`.

Зупинка спливання

- `event.stopPropagation()` — зупиняє подальше спливання події вгору по DOM-дереву.
- `event.preventDefault()` — скасовує стандартну поведінку браузера для події (наприклад, відправку форми або перехід за посиланням).

11. Делегування подій (Event Delegation)

Делегування подій — це патерн оптимізації, при якому один обробник подій призначається на батьківський елемент замість створення обробників на кожному з його дочірніх елементів. Це стало можливим завдяки механізму спливання подій.

```
// Приклад: Обробка кліків у списку з 1000 елементів
const list = document.querySelector('#todo-list');

list.addEventListener('click', function(event) {
  // Перевіряємо, чи клікнули саме по елементу з класом delete-btn
  if (event.target.classList.contains('delete-btn')) {
    const item = event.target.closest('li');
    item.remove();
  }
});
```

12. Практичний випадок: Створення динамічного додатка "Task Manager"

Нижче наведено інтегрований приклад, який об'єднує створення HTML-елементів, маніпуляцію CSS, використання `addEventListener` та делегування подій.

```
// Ініціалізація додатку
document.addEventListener('DOMContentLoaded', () => {
  const taskForm = document.querySelector('#task-form');
  const taskInput = document.querySelector('#task-input');
  const taskList = document.querySelector('#task-list');

  // Обробка відправки форми
  taskForm.addEventListener('submit', (e) => {
    e.preventDefault(); // Запобігаємо перезавантаженню сторінки

    const taskText = taskInput.value.trim();
    if (taskText === '') return;

    createTaskElement(taskText);
    taskInput.value = ''; // Очищення поля
  });

  // Створення елемента завдання
  function createTaskElement(text) {
    const li = document.createElement('li');
    li.className = 'task-item';

    li.innerHTML = `
      <span class="task-text">${escapeHTML(text)}</span>
      <button class="btn-complete">Виконано</button>
    `;
  }
});
```

```

        <button class="btn-delete">Видалити</button>
    `;

    taskList.appendChild(li);
}

// Делегування подій для кнопок у списку
taskList.addEventListener('click', (e) => {
    const target = e.target;
    const taskItem = target.closest('.task-item');

    if (target.classList.contains('btn-delete')) {
        taskItem.remove();
    } else if (target.classList.contains('btn-complete')) {
        taskItem.classList.toggle('completed');
    }
});

// Екранування введеного тексту від XSS
function escapeHTML(str) {
    const div = document.createElement('div');
    div.textContent = str;
    return div.innerHTML;
}
});

```

Підсумок

1. **JavaScript** — це гнучка та динамічна мова програмування, що працює на основі ECMAScript стандарту та однопотокової асинхронної моделі Event Loop.
2. **DOM (Document Object Model)** представляє структуру веб-сторінки у вигляді об'єктного дерева, доступного для вибірки та модифікації за допомогою JS.
3. Ефективна маніпуляція DOM вимагає розуміння різниці між методами вибірки (`querySelector`), розумного управління стилями через `classList`, використання безпечних методів вставки (`textContent`, `append`) та використання патерну **делегування подій** для високої продуктивності додатків.